

浮点验证与规约测试：工作进展汇报

IP 全链路验证 (QCP/Rocq) 与 FloatTest 位级差分测试

QCIP 验证组

2026 年 9 月

目录

1 背景：我们要验证什么

- 验证对象
- 总体进展时间线
- 两条技术路线
- 技术工作一览
- 全称证明与采样定理

2 路线一：QCP 全链路验证

- 验证流水线
- 示例 1：STUseFlag 状态机
- 浮点建模的三代演进
- 示例 2：float_clamp（原生浮点）
- 示例 3：float_store
- 路线一总览

3 路线二：FloatTest 位级差分测试

- 路线二的动机
- FloatTest 流程与 PBT 设计
- 示例 4：ModeConvert_SBM
- 三个技术难点
- 浮点处理技术汇总
- 覆盖规模与证据
- 测试发现的问题与经验

4 总结与展望

- 工作量量化
- 总结
- 未来工作

验证对象：星载控制软件 IP

- **从哪里来：**卫星姿态与轨道控制系统的核心算法模块（C 语言）
 - 姿态解算、陀螺数据处理、轨道计算、岁差章动、喷气推进控制...
 - 工作模式切换判断（ModeConvert 系列）
- **共同特点**
 - **浮点密集：**大量 float64 四则运算、限幅、比较
 - 频繁从结构体/数组中读写浮点数
 - 调用 libm: sin/cos/sqrt, 甚至 asin/atan2/exp
 - 分支多、状态多：一个函数几十路输入输出
- **形式化的必要性：**在轨软件不可维护，浮点误算代价高

总体进展时间线（2026-07 至 2026-08，约 7 周）

时间	路线	进展
07-02	验证	首个 IP 全链路证明完成 (STSUseFlag, 7 条分支全证)
07-09	验证	StructFilter 完成 (第二代建模: float64 当 64 位整数)
07-13	验证	STSTransDataSave 完成 (annotation 迭代 5 轮)
07-14	验证	首个原生浮点验证案例 (float_clamp / float_store)
07-15	验证	WheelFriction 部分验证——浮点抽象的局限暴露
07-24	测试	FloatTest 框架推广: SAM 7 题 + ModeConvert 6 题 + CS 前 2 题
08-05	测试	三角函数破题 : musl sin/cos 双侧确定性移植
08-09	测试	发现 MSVCRT sqrt 1-ulp 误舍入 $\Rightarrow$ sqrt 移植
08-10	测试	姿态 selftest 抓出 6 处共享层错误, 5 题重生成
08-17	测试	iplib 24/24 全部交付 , 36,271 条定理
08-19	测试	7 题证据链刷新 + dashboard 看板上线

- 另有 orbiter-new 批次 35/35 交付 (35,035 条定理); 两条路线并行推进
- 07-14 原生浮点可用、08-05 三角函数解决——两类原本无法处理的程序先后纳入范围

两条技术路线

路线一：QCP 全链路形式化验证

- 在 C 代码上写规约 **annotation**（前置/后置条件）
- 符号执行自动生成**验证条件**（VC）
- 在 Rocq 中人工完成**全称证明**（对所有输入成立）
- 结论为全称命题，但依赖工具链对浮点的支持程度

路线二：FloatTest 位级差分测试

- 把 C 算法**逐行转写成** Rocq 可执行规约
- 用 gcc 跑出真值，在采样输入上**逐比特**比对
- 每条测试向量 = 一条机器检查的 Rocq 定理
- 覆盖证明暂不可行的浮点密集程序

工具链的浮点支持程度决定程序采用哪条路线。

技术工作一览（三个方面）

贡献 1：浮点程序的原生形式化验证机制（路线一）

基于 Flocq 的原生 fp32/fp64 规约类型；用有限性前提在规约中显式处理 NaN/ $\pm\infty$ ；用类型化内存谓词 `store_float` 表达浮点内存写入。完成 `float_clamp` / `float_store` 两个案例。

贡献 2：确定性浮点运行时与位级规约技术（路线二）

musl sin/cos/sqrt 双侧逐行移植，把 libm 变成确定性可复现运算；C 语言比较语义（NaN 为假）在 Rocq 侧精确复刻；混合精度、舍入模式（`mode_NE`）、bit pattern 逐位对齐。

贡献 3：定理化的 PBT 差分测试框架 FloatTest

“测试即定理”：每条向量对应一条由 `vm_compute` 检查的 Rocq 定理；随机 + 定向边界输入设计；阴性自检验证测试有效性；全链路证据固定（种子、快照、新鲜度重放）。

全称证明与采样定理

全称证明（路线一）

一条定理覆盖所有合法输入：

$\forall x\ lo\ hi : Safe(x, lo, hi) \Rightarrow lo \leq clamp(x, lo, hi) \leq hi$

证明由 Rocq 内核检查。

测试定理（路线二）

每个采样点一条定理，各证一次：

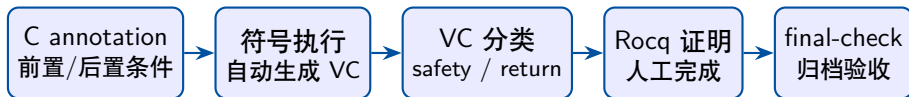
`t_0861 : spec(输入861) = 输出861`

1000 条向量 = 1000 条定理，每条经 Rocq 内核计算验证。逐比特严格，但仅覆盖采样点。

比较

全称证明覆盖所有合法输入；采样定理仅覆盖采样点，但每条结论均由 Rocq 内核检查，严格性高于普通测试。该限制在各 case 报告中均有声明。

全链路验证流水线



- **Annotation**: 用分离逻辑描述“函数执行前内存里有什么 (Require)”和“执行后保证什么 (Ensure)”
- **Safety VC**: 内存安全 (不越界、类型匹配), 多由自动策略完成
- **Return VC**: 功能正确性 (返回值满足后置条件), 人工证明——工作量主体
- **final-check**: 无 Admitted/Axiom 扫描、生成文件新鲜度复核、归档哈希比对

示例 1: STSUseFlag 状态机验证

C 代码 (分支状态机, 7 条路径)

```
if (sSTSAtt.dataSrc == 1) {
    if (sBusSTSObj.update == 3952088175) {
        if (sSTSAtt.stsFlg == 1)
            sSTSAtt.stsFlg = 6;
        else if (sSTSAtt.stsFlg == 2)
            sSTSAtt.stsFlg = 7;
        else if (sSTSAtt.stsFlg == 4)
            sSTSAtt.stsFlg = 8;
        sSTSAtt.useOld = sSTSAtt.stsFlg;
    } else {
        if (sSTSAtt.stsFlg != 3)
            sSTSAtt.stsFlg = sSTSAtt.useOld;
    }
} else {
    sBusSTSObj.update = 0;
}
```

Annotation (分离逻辑规约, 节选)

```
/*@ Require store(&(sSTSAtt.dataSrc), data_src) *
    store(&(sBusSTSObj.update), update) *
    store(&(sSTSAtt.stsFlg), status) *
    store(&(sSTSAtt.useOld), use_old)
    Ensure exists data_src1 update1 status1 use_old1,
        STSUseFlagPost(data_src, update, status, use_old,
            data_src1, update1, status1, use_old1)
    && store(&(sSTSAtt.stsFlg), status1) * ... */
```

- store(p, v): 内存地址 p 当前存储值 v
- *: 分离合取——几块互不重叠的内存
- 后置: 存在一个新状态, 满足后状态关系 STSUseFlagPost

示例 1 (续): 后状态关系规约 (predicate-first)

后状态关系 STSUseFlagPost 的语义 (表格化)

路径条件	status1	update1
数据源有效 $\wedge$ 更新标志命中	1 $\rightarrow$ 6 / 2 $\rightarrow$ 7 / 4 $\rightarrow$ 8	不变
数据源有效 $\wedge$ 标志未命中	恢复为 use_old	不变
数据源无效	不变	清零

7 条分支路径 $\Rightarrow$ 7 个 return VC, **全部证明完成。**

方法论: predicate-first

规约描述“输入状态与输出状态之间的**数学关系**”，而非在 Rocq 中复制 C 的 if-else 控制流。

- 好处: 规约可读、可审查、与实现解耦
- 反模式 (归档中明确记录): 镜像 C 控制流, 证明随分支爆炸

耗时构成

证明命令本身为秒级; 主要耗时在 Rocq 依赖环境编译 (约 285 s/次)。

浮点建模的三代演进

第一代：整数替换

WheelFriction (动
量轮摩擦补偿)

`float64` $\rightarrow$ `uint32`

公式 $k_f \cdot h / cut + vf0 \cdot \text{sgn}(h)$

被抽象为未解释函数

137 个 safety VC 无法经
济证明，全部 Admitted

第二代：不透明整数

StructFilter (结构滤波 + PID)

`float64` $\rightarrow$ `long long`

浮点视为 64 位不透明
字，仅证明整数层规格

全部 VC 证明完成，
但不含浮点语义

第三代：原生浮点

`float_clamp` / `float_store`

QCP v2.0.4 原生

`fp32`/`fp64`，底层 Flocq

浮点比较、有限性、内存
写入全部在证明层表达

- 前两代为工具链不支持浮点时期的过渡方案；第三代 (v2.0.4 起) 开始表达浮点语义
- 同期对证明暂不可行的浮点密集 IP，采用 FloatTest 测试路线 (后半部分)

示例 2: float_clamp 验证 (原生浮点)

带 annotation 的 C 代码 (真实归档文件)

```
/*@ Extern Coq (clampFloatSafe : fp32 -> fp32 -> fp32 -> Prop) */
/*@ Extern Coq (clampFloatPost : fp32 -> fp32 -> fp32 -> fp32 -> Prop)
    */

float float_clamp(float x, float lo, float hi)
/*@ Require clampFloatSafe(x, lo, hi)
    Ensure clampFloatPost(x, lo, hi, __return) */
{
    if (x < lo) return lo;
    if (x > hi) return hi;
    return x;
}

double double_clamp(double x, double lo, double hi)
/*@ Require clampSafe(x, lo, hi)
    Ensure clampPost(x, lo, hi, __return) */
{ /* 同样三条分支 */ }
```

Rocq 侧的规约定义 (真实归档文件)

```
Definition clampFloatSafe (x lo hi : fp32) : Prop :=
  fp32_isFinite x /\ fp32_isFinite lo /\
  fp32_isFinite hi /\ fp32_le lo hi.
```

```
Definition clampFloatPost (x lo hi ret : fp32) : Prop :=
  fp32_ge ret lo /\ fp32_le ret hi.
```

- fp32 是 QCP v2.0.4 的原生浮点类型 (底层 Flocq 库)
- 前置条件: 三者有限且 $lo \leq hi$
- 后置条件: $lo \leq ret \leq hi$

示例 2 (续): NaN 与前置条件设计

若前置条件允许 $x = \text{NaN}$

- C 语言规定: NaN 参与的任何比较均为 false
- 两个分支均不成立, 函数原样返回 NaN
- 后置条件 $lo \leq r \leq hi$ 对 NaN 不成立, 规约不可证

处理

- 前置条件用 `fp32_isFinite` 排除 NaN 与 $\pm\infty$
- 即显式记录假设: 仅在有限输入下保证钳位性质

证明要点

- 2 个函数 $\times$ 3 条分支 = 6 个 return VC, 全部证明
- 规约库只有 4 个谓词 + 4 条引理
- 核心引理: 有限浮点的比较自反 (x 与 x 比较得 Eq)、可交换反转 (交换两边 Lt 变 Gt)
- 无 Admitted, final-check 通过

说明

本仓库首个使用原生浮点类型完成证明的案例; 规约直接表达浮点比较与有限性, 不再将浮点建模为整数。

示例 3: float_store 与类型化内存谓词

验证目标

- $*p = x$; (x 为 float) 在分离逻辑中对应的谓词
- 证明符号执行产生的写法与原生谓词等价:

$$p \# \text{Float} \mapsto x \mid \text{--- store_float } p \ x$$

- 两个案例: 直接写变量、写结构体字段, 各 1 个 return VC, 均证明完成

遗留: 证明完成, 但最终 canonical coqc 验收环节因环境缺失待补跑。

失败反例 (归档于 failing_examples/)

若 annotation 中使用通用 `store(p, x)` 写浮点, 符号执行报错:

Cannot unify types fp32 and Z

原因: 通用 `store` 仅接受 `Z` 类型值, 而 x 为 `fp32`。浮点内存须使用类型化谓词 `store_float`。

路线一总览：6 个 case 的交付状态

Case	功能	Manual VC	状态	备注
STSUseFlag	星敏感器使用标志状态机	7 return	✓ 完成	7 条分支路径全证
StructFilter	结构滤波 + PID 控制器	1 return + 30 safety	✓ 完成	float64 建模为 64 位整数
STSTransDataSave	SRAM 248 字节分段拷贝	2 return	✓ 完成	annotation 迭代 5 轮
WheelFriction	动量轮摩擦力矩补偿	4 return + 137 safety	△ 部分	浮点抽象, safety 全 Admitted
float_clamp	fp32/fp64 钳位函数	2×3 return	✓ 完成	首个原生浮点验证
float_store	float 内存写入谓词	2 return	✓ 证明完成	待补最终 coqc 验收

- 4 个完整证明通过 final-check、1 个部分验证、1 个证明完成待验收
- Annotation 是主要迭代点：STSTransDataSave 迭代 5 轮得到正确的分段规格
- 遗留：WheelFriction 的 137 个 Admitted 等原生浮点能力重做；float_store 待补验收

路线二的动机：QCP 浮点支持的限制

目标程序的典型形态

```
// 从结构体读浮点数组 -> 浮点运算 -> 写回  
tmpCp = Angle2C321(0, 0, sAtt.Psi_DA);  
Cro   = MatrixMulti333(Cbiasp, tmpCp);  
qri   = C2Q(Cro);           // 含三角函数与 sqrt
```

当时的工具链限制

- QCP 无法把内存中读出的 64 位数据还原为浮点（只当整数处理）
- 这批程序没有 annotation，且大量依赖 libm 三角函数
- 符号执行全链路无法完成

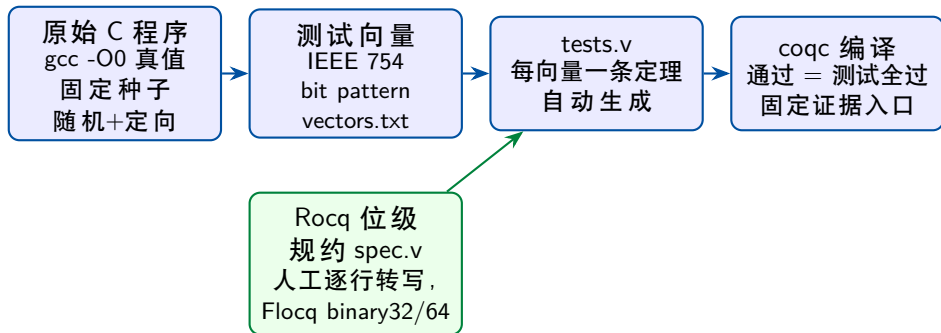
FloatTest 的思路

全称证明暂不可行时，检验一个较弱但可机器判定的性质：

Rocq 规约与 C 实现在采样输入上是否逐比特一致

方法：规约逐行人工转写，与 gcc 真值逐比特比对，每个采样点形成一条 Rocq 定理。

FloatTest 流程：测试即定理



- **真值**：原始 C 用 gcc 编译（关闭 FMA 收缩），固定种子生成随机 + 定向边界输入（阈值恰等、NaN、 $\pm\text{Inf}$ 、 ± 0 、饱和、计数器回绕等 7-13 类）
- **比对粒度**：输入输出一律按 64 位 bit pattern 比较，不作近似，1 比特差异即判失败
- **阴性自检**：每个 case 附 1 条故意改错期望值的定理，预期编译报错，用于验证工具链有效性

PBT 设计：六个关键决策

PBT 要素	FloatTest 的设计	设计理由
被测性质	采样输入上 C 输出 $\equiv$ 规约输出 (逐比特, 约定 NaN==NaN)	浮点不作近似比较, 1 ulp 差异即判失败
Oracle (真值)	原始 C + gcc -O0 (关 FMA 收缩) + musl 确定性移植	宿主 libm 不可信 (sqrt 误舍入实证)
输入生成	固定种子随机 + 7-13 类定向边界 (阈值恰等、NaN、 $\pm\text{Inf}$ 、 ± 0 、饱和、回绕)	随机覆盖一般行为, 定向命中浮点奇异点
检查方式	vm_compute; reflexivity——内核计算的定理	检查由 Rocq 内核执行, 不依赖外部脚本的判断
有效性验证	每题 1 条阴性自检: 故意改错, 预期报错	变异测试思想: 确认测试可检出错误
可复现性	固定种子 + SHA-256 快照 + 新鲜度重放 + 固定 coqc argv	任何人任何时间重跑结论一致

与普通 PBT (如 QuickCheck) 的差异

普通 PBT 的通过记录是测试日志; FloatTest 的每条通过向量都对应一条 Rocq 定理, 且规约文件可在后续证明工作中直接复用。

示例 4: ModeConvert_SBM 测试流程

第 1 步: 被测 C 代码 (全部逻辑)

```
// 星时超过注入时刻 + 时延 -> 转入 EIM 模式
if ((m_starTime - t0) >= dt_OrbitInject)
    m_WorkMode = WKMD_EIM; // 0x11
// 否则工作模式不变
```

第 2 步: 参考驱动生成向量 (vectors.txt 真实两行)

```
4694428428020873060 13909875606412569204
    4661949024290739460 20 17
4683088663683732336 4683088663683732336 4607182418800017408
    3 3
```

列含义: starTime、t0、dt (都是 64 位 bit pattern)、旧模式字 → 新模式字

第 3 步: 人工转写的 Rocq 规约 (spec.v 真实全文核心)

```
Definition modeConvert_SBM_fun
  (starTime t0 dt : fp64) (workMode : Z) : Z :=
  if c_ge64 (fp64_sub starTime t0) dt
  then WKMD_EIM_Z (* 17 = 0x11 *)
  else workMode.
```

第 4 步: 每条向量变成一条定理 (tests.v 真实片段)

```
Example t_0002 :
  modeConvert_SBM_fun (f64 4683088663683732336)
    (f64 4683088663683732336) (f64 4607182418800017408) 3 = 3.
Proof. vm_compute. reflexivity. Qed.
```

vm_compute 由 Rocq 内核计算左端, 与右端相等则定理成立。本例 starTime = t0, 差值 $0 < dt$, 条件为假, 模式保持 3。

位级测试的四个关键概念

bit pattern (位模式)

浮点数在内存中为 64 个比特；向量中的大整数即这些比特的整数值。按位比较严格于任何误差界比较。

vm_compute: 内核级计算

Rocq 内核的虚拟机实际执行规约函数。“通过”的含义是内核确认等式两端相等，而非外部脚本的判断。

c_ge64: 复刻 C 的 NaN 语义

C 规定 NaN 的比较一律为假。Flocq 的原始比较遇 NaN 返回“无法比较”，因此封装 c_ge64：无法比较 $\Rightarrow$ false，与 C 完全一致。

不做 extraction 的原因

规约为人工逐行转写，转写本身可能出错，差分测试用于检出此类错误；自动提取则失去对照意义。

难点 1: 三角函数 (15 个 case 曾无法处理)

问题

- Flocq 仅定义 IEEE 四则运算与 sqrt 的舍入语义, 不含 sin/cos
- 各 libm 实现的三角函数结果可能不同, 缺少确定的真值口径
- 姿态计算类 case 普遍调用三角函数, 15 题受阻

解决 (2026-08-05): libm 运算确定化

- 将 musl libc 的 sin/cos (含参数约减 `__rem_pio2`) 逐行移植两份: C 侧 (编译时替换系统 libm) 与 Rocq 侧 (FloatTrig.v)
- 两侧执行同一算法, 三角函数成为确定性、可复现的运算

移植质量自测

- 独立 selftest: 3176 条向量逐比特一致
- 与 msvcrt libm 对比: 常规值 0 ulp 差, 大参数约 21 ulp (musl 更准)
- sqrt 同样移植 (musl 纯整数 Goldschmidt 迭代), 自测 2073 条向量

残余边界

asin/atan2/exp 未移植: 依赖它们的子函数用“打桩 + 输入注入”隔离——桩的输出直接作为输入注入, 下游真实计算仍逐比特比对, 并在每题 README 置顶声明

难点 2: 宿主 libm 的 sqrt 误舍入

事件经过 (CS_TrgtAtt_NWM_USU, 2026-08-09)

- ① 1025 条向量中 **t_0861** 一条失败: 输出四元数 4 个分量各差 ± 1 ulp
- ② 逐层排查, 定位到 sqrt: 本机 MinGW gcc 链接的老 MSVCRT 库存在罕见误舍入 (x87 FSQRT 双舍入残留)
- ③ 实例: $x = 0x1.de63fa81fe9bcp+0$
 - MSVCRT 给出 ...**034** (错)
 - 正确舍入 ...**033** (Rocq 侧 Flocq Bsqrt 的结果)
- ④ 此前三个 case 约 3000 条向量未覆盖该输入, 属偶然

处理与结论

宿主 libm 不能作为真值来源, 真值必须由确定性实现产生。据此移植 musl sqrt, 受影响 4 题重跑全部通过。

ulp 的含义

ulp (unit in the last place) 是浮点末位一个比特代表的值, 即浮点表示的最小差异; 逐比特比对可检出该量级的错误。

难点 3：共享规约层的独立自测

背景

- 姿态类 case 共用一批基础算法：六种转序的 Angle2C（姿态角 $\rightarrow$ 旋转矩阵）、矩阵乘、四元数运算（QMulti / Q2C / C2Q）
- 提取为 C/Rocq 两侧共享库，避免每个 case 复制展开
- 风险：共享层单点错误传播到所有 case

独立自测 (attitude selftest)

- 4 个精确锚点：单位阵、绕 X/Y/Z 轴转 π
- 6 种转序 $\times$ 10000 条随机，从基础矩阵相乘独立重算对照

发现的问题

- 4 个 Angle2C 第一轴符号错误
- 2 个 C2Q 对称项下标错误
- EIM、AHM_USU、P2P、OCM 等 5 个受影响 case 重新生成并通过

经验

修改共享库后必须重跑所有受影响 case，避免留下“指向旧版本摘要的通过记录”。

浮点处理技术汇总（问题 → 处理方式）

浮点问题	处理方式
IEEE 舍入语义	Flocq Bplus/Bminus/Bmult/Bdiv (mode_NE 最近偶数舍入) 与 gcc -frounding-mode=nearest 逐位对齐
内存中的浮点	64 位 bit pattern 注入/读出 (b64_of_bits); 证明侧用类型化谓词 store_float
NaN 的比较语义	封装 c_ge64 等: 不可比较 $\Rightarrow$ false, 精确复刻 C
证明层的 NaN/ ∞	前置条件 isFinite 显式排除, 把假设写进规约
三角函数 (Flocq 不支持)	musl sin/cos 双侧逐行移植 (含参数约减), 自测 3176 向量逐比特一致
sqrt 真值分歧	musl sqrt (纯整数 Goldschmidt) 移植 + Flocq Bsqrt 正确舍入
asin/atan2/exp 未移植	打桩 + 输入注入, 边界在 README 置顶声明
混合精度	按 C 转换规则显式建模 binary64 $\leftrightarrow$ binary32 转换; f 后缀字面量逐行核对

- 上述对策基于同一套 Flocq 语义, 证明与测试两条路线共用, 后续做全称证明时可直接复用

测试覆盖规模与结果

批次	Case 数	向量/定理数	通过率	内容
iplib	24/24	36,271	100%	6 个模式切换 + 18 个姿态/轨道计算
orbiter-new	35/35	35,035	100%	轨道器批次, 各 1000 条定向向量
SAMCodeSynthesis	8/9	7,000	100%	速率/姿态控制等, 最重 1 题未做
cfg_target (早期试点)	12	431 正例	100%	框架首批验证对象
合计	79	≈78,700 条定理	100%	全部无 Admitted/Axiom

- 典型每 case 1000 条随机 + 定向边界向量; 最重 case 单题 5013 条
- 千条向量全链路约 10 秒; 固定种子, 完全可复现
- 每题 1 条阴性自检 (故意改错) 全部正确报错
- 全链路 SHA-256 快照绑定源码/规约/向量/工具

iplib 批次：24 个 case 与证据看板

两个程序家族

- **ModeConvert** (6 题)：工作模式切换。例：EIM 题以模式字为下标读 fp64[14] 定时器数组，超时转 NWM
- **CS 系列** (18 题)：陀螺最小二乘/直接求逆、姿态控制喷气调度、轨道计算、IAU1976 岁差 + IAU1980 章动、点对点机动规划...
- 最重一题 CS_TrgtAtt_AHM_USU：104 个输入 → 44 个输出，9 段混合轨迹角速度规划

共享姿态规约层

- 六种 Angle2C、矩阵乘、四元数运算，C/Rocq 双侧共享

证据看板 (dashboard)

- 静态网页索引：C 源码、规约声明、 $C \leftrightarrow Rocq$ 列映射、示例向量、复现命令
- 7 个 case 已刷新完整证据链（固定 coqc 检查 + 向量新鲜度逐字节比对 + 阴性控制）
- 其余 17 个为历史通过、待补证据刷新（非测试失败）
- 24/24 规约与测试无 Admitted/Axiom

测试发现的问题

- ❶ 宿主 libm 误舍入：MSVCRT sqrt 存在 1-ulp 误舍入 $\Rightarrow$ 促成 musl sqrt 移植（详见难点 2）
- ❷ 姿态公式转写错误：4 个 Angle2C 符号错误 + 2 个 C2Q 下标错误，被独立 selftest 抓出（详见难点 3）
- ❸ 驱动程序 UB：GyroPick 的参考驱动 printf 格式串多写 2 个 %u，打印出垃圾列，被向量生成器的列数断言拦截
- ❹ 源码笔误：CS_IRES_Modify 循环误用未初始化索引 i（gcc 警告 + 测试边界双重记录）
- ❺ 混合精度陷阱：binary64 实参传给 float 形参，要显式建模 64 $\rightarrow$ 32 $\rightarrow$ 64 两次转换；0.9231f 与 0.9231 精度不同，此类转写偏差可由差分测试即时检出

小结

这些问题分属三类：环境（libm）、规约转写（Angle2C/C2Q）、被测代码及其驱动（UB、笔误）——差分测试对三类都有效。

测试路线：经验沉淀

方法论

- ① 决定性难度因素**不是代码行数**，而是是否依赖 libm 三角/反三角/exp
- ② 真值口径固定为“原始 IP + musl 确定性移植”，结论可复现
- ③ 确定性基础运算提成两侧共享层，避免复制展开导致符号/下标漂移
- ④ 修改共享库后必须重跑所有受影响 case

工程纪律

- ⑤ 正式证据只接受固定 `coq_tooling.py` check 在隔离 workspace 的固定 argv 结果
- ⑥ 每题 README 置顶声明重建约定、打桩边界与未覆盖范围
- ⑦ 全链路 SHA-256 快照绑定：C 源码、规约、驱动、向量、发射器

定位

FloatTest 是 QCP 全链路的**补充**而非替代：在证明暂时不可达的浮点程序上，用采样定理给出机器检查的一致性结论；规约库可在将来做全称证明时复用。

工作量量化

路线一：验证（6 case，约 2 周）

- 人工 Rocq 证明 $\approx 1,330$ 行 (proof_manual 合计)
- 人工数学规约库 ≈ 630 行
- 人工证明 return VC 22 个 + safety VC 30 个 (含拆分约 90 条引理)
- 最大单 case: StructFilter 生成 goal 9,749 行
- annotation 最多迭代 5 轮 (STSTransDataSave)

路线二：测试（79 case，约 5 周）

- 人工逐行转写规约 $\approx 4,300$ 行 (三批次 spec.v + 共享库)
- 生成并编译通过定理 $\approx 78,700$ 条 (tests.v 合计 ≈ 47 万行)
- 测试向量 78,271 条，每题 7-13 类定向边界
- musl sin/cos/sqrt 双侧移植 3 套；独立自测向量 6 万+ 条
- 中文 README 边界声明 $\approx 2,600$ 行；静态 dashboard 1 套

合计：85 个 case 交付，证明与测试两条线共覆盖 IP 库的主要浮点程序。

总结：三方面技术工作，两条路线

技术贡献

- ① **浮点原生验证机制**：fp32/fp64 规约 + 有限性前提 + 类型化内存谓词，规约直接表达浮点语义
- ② **确定性浮点运行时与位级规约**：musl 移植 + C 语义复刻，解决“浮点没有可信真值”的根本难题
- ③ **定理化 PBT 框架 FloatTest**：测试即定理 + 阴性自检 + 证据固定，采样测试具有机器检查的结论

证明可达的程序做全称证明；证明暂不可达的浮点程序，用逐比特的定理化测试覆盖。

完成情况

- 路线一：6 case，4 个完整证明，三代浮点建模演进
- 路线二：79 case，约 **78,700 条定理 100% 通过**，零 Admitted/Axiom
- 实证发现的问题：宿主 libm 误舍入、规约转写错误、驱动 UB

验证路线

- 用原生浮点能力重做 WheelFriction 等早期抽象 case, 补齐 137 个 Admitted
- float_store 补跑 canonical coqc 验收
- 沉淀外部函数规格库; 循环不变式自动生成

测试路线

- 补齐 17 个 archived case 的最新证据链
- 完成遗留的最重 case (SAM GyroAttiDetermine: 矩阵求逆 + fp64 floor)
- asin/atan2/exp 的确定性移植, 消除打桩边界
- **测试** → **证明衔接**: 已验证的 Flocq 规约库可直接复用于全称性定理 (如限幅输出界)

两条路线的衔接点: FloatTest 规约库 + QCP 原生浮点支持, 为浮点 IP 的全称证明提供基础。

谢谢！

欢迎讨论与指正